Министерство Образования и Исследований Республики Молдова

Технический Университет Молдовы

Факультет Вычислительной Техники, Информатики и Микроэлектроники

Департамент Программной Инженерии и Автоматики

Отчет

по лабораторной работе Nr.4.

по дисциплине: PAM

Тема: Управление состоянием и Repository

Выполнил ст. гр. TI-236 Manolopoulos Aristeidis

Проверил Leah Arcadie

Кишинев 2026

1. Цель работы и продолжение проекта

Тема проекта: собственная тема «TradeArena — биржевой симулятор» (финансы / образование). Работа продолжает приложение ЛР2 и ЛР3: рынки США, Индии и Европы, виртуальный портфель и учебные ордера со стартовым балансом $10 000.

Цель ЛР4 — отделить экраны от источника данных, ввести общее состояние на ChangeNotifier и Provider, реализовать Repository с результатами Ok/Err и связать торговые формы с записью. Изменение из одного экрана должно немедленно отражаться во всех подписанных вкладках.

Источником пока служит память с демонстрационными данными. Настоящая авторизация и backend относятся к ЛР5. Навигация go_router, формы, темы Material 3 и история Git сохранены.

Установка пакета и структура изменений

```
flutter pub add provider
flutter pub get

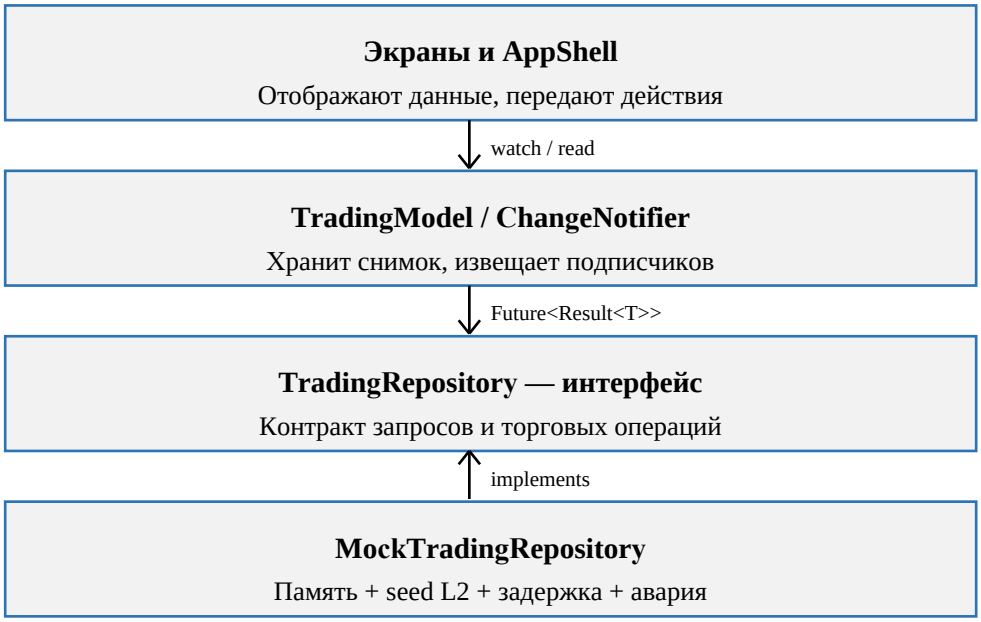
# pubspec.yaml
provider: ^6.1.5+1
```

Первый коммит ЛР4 — 8e6807b «L4: добавлен пакет provider». pubspec.lock фиксирует версию пакета и транзитивную зависимость nested.

```
lib/
  main.dart                        # выбор источника, Provider
  data/models.dart                 # сущности, draft, снимок
  data/result.dart                 # sealed Result / Ok / Err
  data/trading_repository.dart      # интерфейс CRUD
  data/mock_trading_repository.dart # память и торговые операции
  models/trading_model.dart         # ChangeNotifier
  widgets/data_view.dart            # loading / error / data
  widgets/app_shell.dart            # вкладки и Consumer badge
  screens/                         # подписанные экраны
```

2. Архитектура приложения

UI знает модель состояния, модель знает только интерфейс хранилища, а mock-реализация единственная импортирует исходный mock_data.dart. Схема ниже построена для TradeArena.



Назначение слоёв и переход на ЛР5

Экраны показывают данные и передают действия. TradingModel хранит согласованный снимок и вызывает notifyListeners. TradingRepository задаёт контракт. MockTradingRepository хранит данные в памяти и имитирует сетевые запросы.

На ЛР5 добавляется HTTP-реализация TradingRepository, а main.dart выбирает её вместо mock. Экраны и модель состояния сохраняются; демонстрационный переключатель аварии удаляется.

Почему один снимок для торговли

Покупка одновременно меняет деньги, позицию и историю. Поэтому операции записи возвращают Result<TradingSnapshot>: подписчики получают все изменения вместе. Дополнительный запрос после успешной записи не нужен и не может скрыть уже исполненную сделку сетевой ошибкой.

Контракт Repository и ожидаемые ошибки

Ошибки запроса представлены значением Err, а не исключением в UI. sealed Result позволяет исчерпывающе обработать обе ветки через switch.

```
/// Ожидаемая ошибка — значение, которое вызывающий код обязан обработать.
sealed class Result<T> {
  const Result();
}

final class Ok<T> extends Result<T> {
  const Ok(this.value);
  final T value;
}

final class Err<T> extends Result<T> {
  const Err(this.msg);
  final String msg;
}
```

CRUD ордеров и сброс портфеля

```
abstract interface class TradingRepository {
  Future<Result<TradingSnapshot>> fetchAll();
  Future<Result<Order>> fetchById(String id);
  Future<Result<TradingSnapshot>> create(OrderDraft draft);
  Future<Result<TradingSnapshot>> update(String id, OrderDraft draft);
  Future<Result<TradingSnapshot>> delete(String id);
  Future<Result<TradingSnapshot>> resetPortfolio();
}
```

fetchById возвращает ошибку для неизвестного id. create присваивает id и исполняет либо ставит заявку в ожидание. update заменяет ожидающую заявку с повторной проверкой резервов. delete выполняет логическое удаление — отмену с сохранением истории. Исполненный или уже отменённый ордер изменить и отменить нельзя.

Возврат снимка вместо отдельного Order или void — адаптация общего CRUD-примера к торговой транзакции. Обновление заявки реализовано в контракте и модели, проверено тестами; отдельная форма редактирования в существующем интерфейсе не добавляется.

Mock-хранилище: память, задержка и авария

Каждый публичный метод ждёт 800 мс и проверяет simulateError. При аварии возвращается Err с сообщением; торговая запись не выполняется. Начальные списки копируются из seed.

```
Future<Result<TradingSnapshot>> fetchAll() async {
  await _wait(); // Future.delayed(latency), по умолчанию 800 мс
  if (simulateError) return const Err(_networkError);
  _expireDayOrders();
  return Ok(_snapshot());
}

Future<Result<TradingSnapshot>> create(OrderDraft draft) async {
  await _wait();
  if (simulateError) return const Err(_networkError);
  _expireDayOrders();
  return _place(draft);
}
```

Проверки до изменения данных

Хранилище повторно проверяет id акции, положительное целое количество, конечную лимитную цену, свободные деньги и акции. Проверка формы улучшает обратную связь, но не заменяет доменную проверку. Между окончательной проверкой и записью нет await: параллельные заявки не могут потратить один и тот же резерв.

Неизменяемость ответа

```
TradingSnapshot({required List<Order> orders,
  required List<Position> positions, /* остальные поля */})
  : orders = List.unmodifiable(orders),
    positions = List.unmodifiable(positions);

// Вложенные symbols и история цены также защищены от записи.
```

Списки снимка, тикеры новостей и точки графика доступны только для чтения. Все изменения проходят через методы Repository и модели, поэтому обход notifyListeners невозможен.

3. Общее состояние и подписки экранов

TradingModel получает интерфейс через конструктор. Он не создаёт mock самостоятельно и не импортирует виджеты. Локальные поля поиска, фильтров и формы остаются в setState.

```
class TradingModel extends ChangeNotifier {
  TradingModel(this._repository);
  final TradingRepository _repository;
  TradingSnapshot? _data;
  bool _loading = false;
  String? _error;

  List<Order> get orders => _data?.orders ?? const [];

  Future<void> load() async {
    if (_loading || _saving || _disposed) return;
    _loading = true;
    _error = null;
    notifyListeners();
    switch (await _repository.fetchAll()) {
      case Ok(:final value): _data = value;
      case Err(:final msg): _error = msg;
    }
    _loading = false;
    if (!_disposed) notifyListeners();
  }
}
```

watch, read и Consumer

```
// DataView.build: подписка на изменения
final model = context.watch<TradingModel>();

// Повторная загрузка: действие в callback
onRetry: () => context.read<TradingModel>().load(),

// AppShell: только индикатор ожидающих заявок
Consumer<TradingModel>(
  builder: (context, model, child) => Badge(
    isLabelVisible: model.pendingCount > 0,
    label: Text("${model.pendingCount}"), child: child!),
  child: const Icon(Icons.pie_chart_outline),
)
```

Корень композиции и три состояния

```
void main() {
  final repository = MockTradingRepository();
  runApp(MultiProvider(providers: [
    Provider<TradingRepository>.value(value: repository),
    Provider<MockTradingRepository>.value(value: repository),
    ChangeNotifierProvider(
      create: (_) => TradingModel(repository)..load()),
  ], child: const TradeArenaApp()));
}
```

Provider с конкретным mock нужен только переключателю аварии. Жизненным циклом модели управляет ChangeNotifierProvider через create. MaterialApp.router и ветви вкладок из ЛР3 остаются прежними.

Общий DataView

```
final model = context.watch<TradingModel>();
if (model.isLoading) {
  return const Center(child: CircularProgressIndicator());
}
if (model.error != null) {
  return ErrorView(message: model.error!,
    onRetry: () => context.read<TradingModel>().load());
}
return builder(context, model); // данные
```

Рынок, портфель, новости, рейтинг, профиль, детали и форма используют общий компонент для загрузки и ошибки. ErrorView всегда содержит сообщение и «Повторить». Если данные загрузились, но id акции не найден, детали и форма показывают «Акция не существует», а не вечный spinner. В профиле переключатель аварии находится вне DataView и доступен даже после неудачной загрузки.

Три состояния главного экрана

Загрузка данных

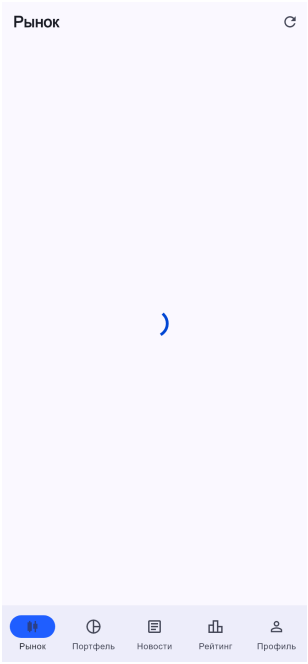


Рисунок 1

После открытия рынка модель находится в isLoading. Виден CircularProgressIndicator. Задержка mock-хранилища делает ожидание наблюдаемым. Снимок получен во время запроса, до публикации данных.

Ошибка с повторной попыткой

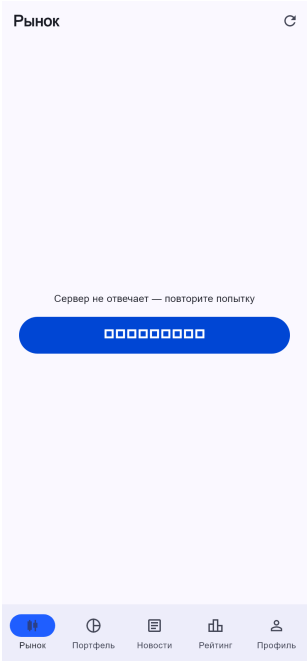


Рисунок 2

При включённой аварии запрос завершается Err. Вместо списка показаны сообщение «Сервер не отвечает — повторите попытку» и кнопка «Повторить». После отключения аварии повторный запрос возвращает данные без перезапуска.

Успешная загрузка и общие данные

Данные рынка после retry

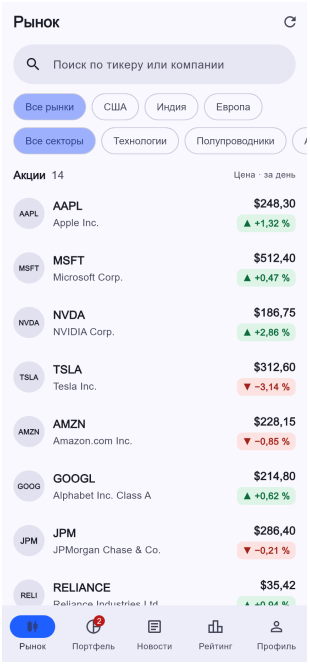


Рисунок 3

Загружены 14 акций. Поиск и фильтры рынка и сектора сохранены с ЛР3, но исходный список теперь приходит из TradingModel. Секторы вычисляются по загруженным акциям. Иконка обновления повторяет запрос через модель.

Рейтинг на основании текущего портфеля

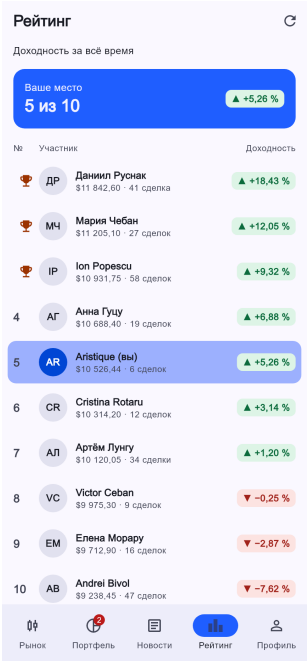


Рисунок 4

Участник Aristique рассчитывается из текущей общей стоимости и числа исполненных ордеров. Рейтинг пересортировывается после каждой операции. Фактическое число демонстрационных участников — 10; прежнее неподтверждённое число 128 удалено.

4. Торговые операции и согласованность

Рыночный ордер исполняется сразу по текущей демонстрационной котировке. Лимитная покупка исполняется, когда цена не выше лимита; продажа — когда не ниже. В другом случае создаётся ожидающий ордер. Автоматического изменения котировок и фонового исполнения в этой лабораторной нет.

Резервы и округление

Ожидающая покупка резервирует сумму и комиссию, ожидающая продажа — количество акций. Полный денежный остаток остаётся частью стоимости портфеля, а доступные для новых покупок средства равны остатку за вычетом резерва. Суммы и комиссия округляются до цента.

```
double money(double value) => (value * 100).roundToDouble() / 100;
double orderFee(double amount) => money(amount * 0.001);

// В исходном портфеле:
// деньги: 1241.85; резерв AMZN: 3 * 215.00 + 0.65 = 645.65
// свободно: 596.20

// Рыночная покупка одной TSLA:
// сумма: 312.60; комиссия: 0.31; к оплате: 312.91
// остаток: 928.94; свободно: 283.29
// общая стоимость: 10526.44 (уменьшилась на комиссию)
```

Изменения позиции и история

Покупка добавляет позицию или увеличивает её количество. Средняя стоимость учитывает прежнюю стоимость, сумму покупки и комиссию. Продажа уменьшает количество без изменения средней цены остатка; нулевая позиция удаляется. Для новых продаж сохраняется реализованный результат, из которого профиль выбирает лучшую сделку.

Дневные лимитные заявки отменяются с наступлением следующей календарной даты при обращении к хранилищу. Заявки «до отмены» продолжают ожидать. Ошибка записи не меняет существующий ордер и не освобождает его резерв.

Мультиэкранная покупка: портфель до и после

Портфель до сделки

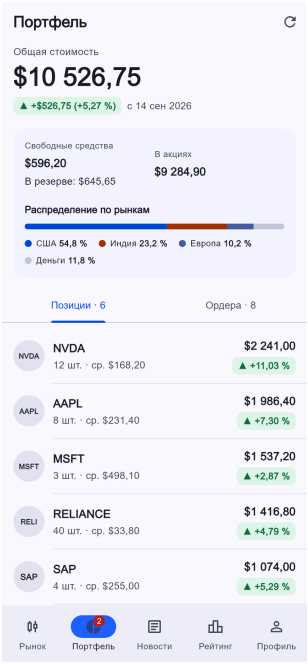


Рисунок 5

До покупки в портфеле 6 позиций и 8 ордеров. Общая стоимость $10 526,75, доступные средства $596,20. Ещё $645,65 зарезервированы исходной лимитной покупкой AMZN. Два ожидающих ордера отражены в badge.

Портфель после покупки одной TSLA

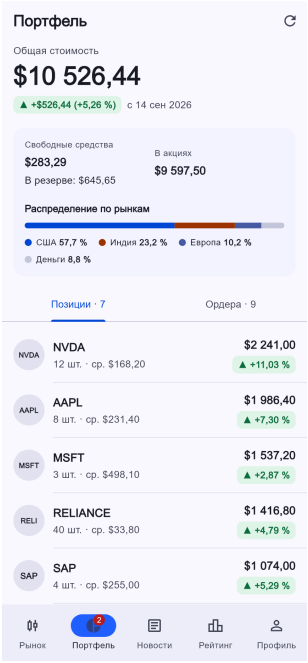


Рисунок 6

Позиции выросли до 7, история — до 9 ордеров. Свободные средства уменьшились до $283,29, стоимость акций выросла до $9 597,50. Общая стоимость $10 526,44 учитывает комиссию $0,31. Портфель обновился автоматически, без ручного запроса после возврата из формы.

Новая позиция и детали акции

TSLA появилась в списке позиций

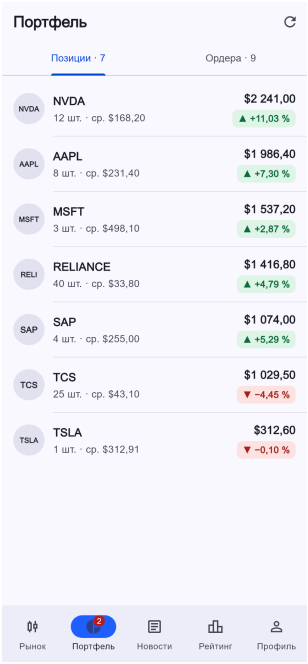


Рисунок 7

После прокрутки портфеля видна новая строка TSLA. Покупка одной акции по $312,60 с комиссией $0,31 дала среднюю стоимость $312,91. Список не получает результат через pop и не перезагружается: его обновляет notifyListeners общей модели.

Детали после успешной покупки

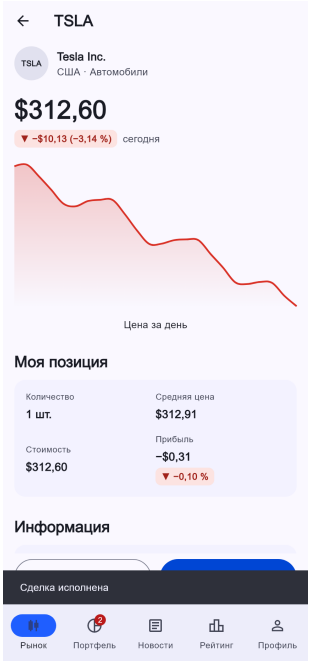


Рисунок 8

После успешной отправки форма закрывается, а карточка TSLA уже читает обновлённую позицию. Сообщение об исполнении относится к выполненной записи, а не к имитации из ЛР3. Та же модель обслуживает портфель, профиль, новости и рейтинг.

5. Связь формы с моделью и Repository

Форма, контроллеры и локальная валидация ЛР3 сохранены. Submit строит OrderDraft, вызывает create модели и дожидается результата. Модель записывает через интерфейс Repository, принимает снимок и извещает подписчиков.

```
Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) return;
  final model = context.read<TradingModel>();
  final error = await model.create(OrderDraft(
    assetId: widget.assetId, side: _side, type: _type,
    quantity: _quantity, limitPrice: _limitPrice,
    untilCancelled: _untilCancelled));
  if (!mounted) return;
  if (error != null) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(error)));
    return; // поля остаются заполненными
  }
  // Успех: снимок уже опубликован для всех экранов.
  context.pop();
}
```

Ожидание отправки и защита данных

Пока запрос выполняется, кнопка отправки отключена, показан индикатор, поля не принимают повторные действия. Модель дополнительно блокирует одновременные записи и перезагрузку во время записи. Проверка mounted защищает UI после закрытия экрана во время запроса.

После неудачного submit данные в контроллерах сохраняются. Для повторной попытки нужно устранить причину ошибки и отправить форму ещё раз. Вход и регистрация остаются на локальной проверке полей до backend и авторизации на ЛР5.

Форма: заполнение и ошибка записи

Валидная форма рыночной покупки

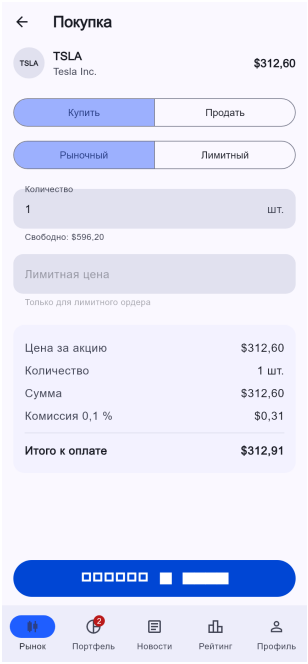


Рисунок 9

Покупка одной TSLA: цена $312,60, комиссия $0,31, итог $312,91. Проверка доступных средств учитывает уже ожидающие лимитные ордера. Для исполняемого лимита расчёт использует более выгодную текущую цену.

Сбой не очищает введённые данные

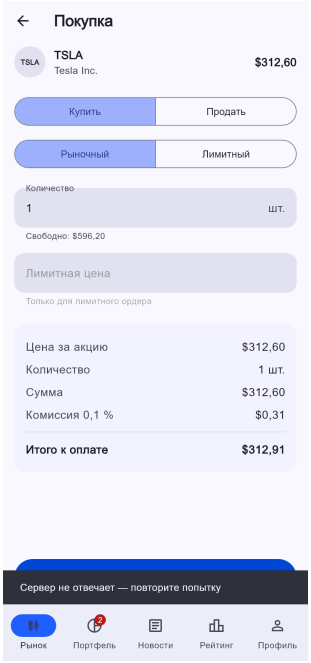


Рисунок 10

При включённом simulateError появляется SnackBar с сообщением Repository. Количество 1 остаётся в поле, форма не закрывается и позиция не создаётся. После выключения аварии тот же ввод успешно приводит к покупке, показанной на предыдущих рисунках.

6. Обратная связь и подтверждение действий

Ожидающий ордер и индикатор

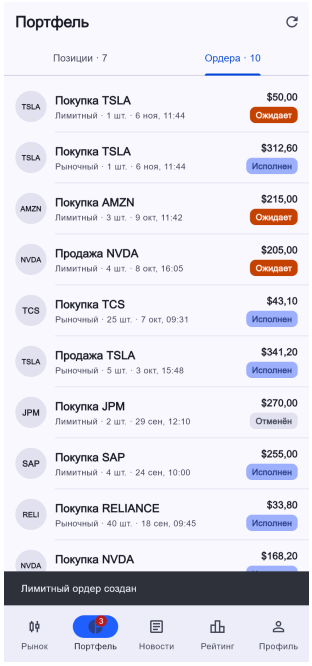


Рисунок 11

Лимитная покупка одной TSLA по $50 не исполняется при текущей цене $312,60. Заявка появляется в истории со статусом «Ожидает», а badge увеличивается с 2 до 3. Резерв новой заявки — $50,05.

Подтверждение отмены

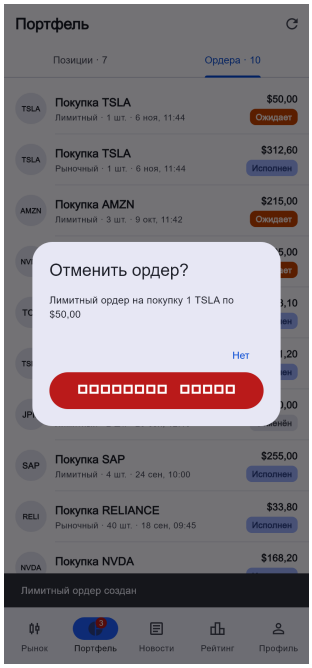


Рисунок 12

Нажатие на ожидающую заявку открывает диалог из ЛР3. Отказ закрывает диалог без записи. Подтверждение вызывает cancel модели, который обращается к delete Repository. На время операции повторная отмена отключена.

Отмена, переключатель аварии и сброс

Результат отмены

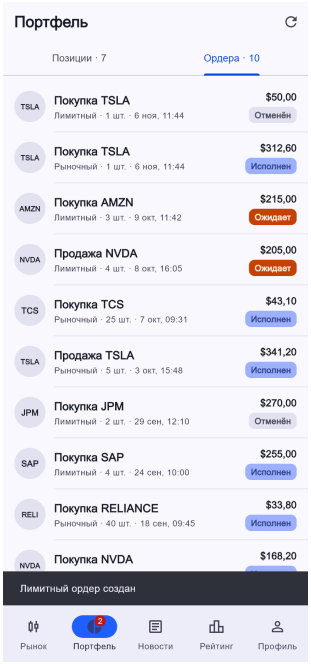


Рисунок 13

Ордер остаётся в истории со статусом «Отменён». Badge снова показывает 2; резерв $50,05 освобождён. Статус и индикатор меняются через общее состояние, а SnackBar подтверждает результат уже завершённой операции.

Профиль и демонстрация ошибки

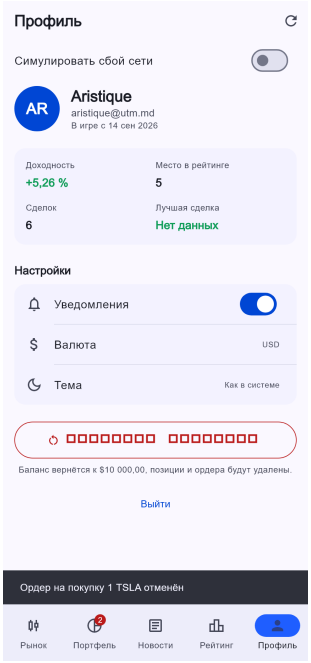


Рисунок 14

Переключатель «Симулировать сбой сети» меняет только флаг mock-источника. Для демонстрации загрузки после включения нужно нажать обновление. Чтобы восстановить данные, выключить переключатель и нажать «Повторить». Переключатель остаётся доступным при ошибке.

Сброс портфеля и проверка реализации

Портфель после подтверждённого сброса

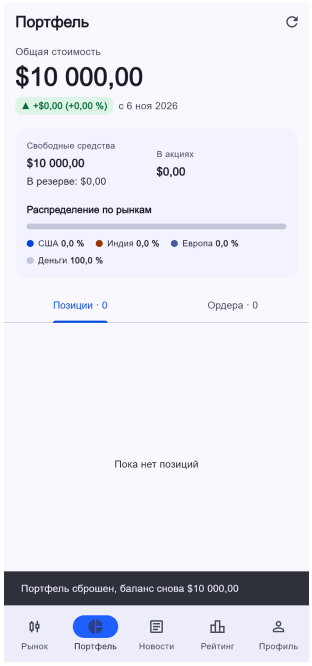


Рисунок 15

Баланс возвращается к $10 000, позиции и ордера удалены. Счётчик ожидающих заявок исчезает; рейтинг и новости также читают новое состояние. Пустой список показывает понятное сообщение.

Автоматические проверки

Выполнены 19 тестов: 16 проверок Repository и модели, два пользовательских сценария через интерфейс и проверка запуска. Покрыты комиссии, средняя стоимость, полная продажа, резервы, выгодный лимит, CRUD, сброс, аварии всех методов, неизменяемость снимков, параллельные заявки, истечение дневного ордера, retry и неизвестный id.

Снимки отчёта получены настоящим рендером Flutter в widget-тестах при размере 430 × 932 логических пикселя. Это тестовое окружение, а не снимки телефона или эмулятора.

```
flutter analyze --no-pub
flutter test --no-pub
flutter build web --release --no-pub

# Обновление снимков реального UI для отчёта:
flutter test --no-pub test/ui_flow_test.dart \
  --dart-define=CAPTURE_L4=true
```

7. История Git и непрерывность

Репозиторий: https://github.com/aristique/Trade-Arena. Ниже приведена полная история от ЛР2 до текущего этапа ЛР4, git log --oneline, от новых изменений к созданию проекта. Код и проверки отчёта соответствуют коммиту ea028b6; сам отчёт добавляется следующим коммитом.

```
ea028b6 L4: тесты торговли и UI, исправление прокрутки портфеля, документация
e8bef65 L4: профиль, реальный сброс и переключатель сетевого сбоя
ad2de2b L4: рейтинг пересчитывается из текущей стоимости портфеля
76e62e2 L4: новости зависят от позиций и выбранного рынка
9b5d7af L4: портфель и badge обновляются после сделок и отмены ордеров
a87c51e L4: форма ордера сохраняет сделку через Repository
c23cac3 L4: карточка акции через модель, обработка неизвестного id
b6d2d07 L4: рынок читает общее состояние, загрузка и ошибка с повтором
3945773 L4: Repository с Result и атомарными торговыми операциями, общее состояние Provider
8e6807b L4: добавлен пакет provider
59640ad На рынке заработали поиск по тикеру и названию и фильтры по рынку и сектору
062860b Ожидающий ордер в портфеле теперь можно отменить, перед отменой спрашивается подтверждение
a8e3b92 Сброс портфеля ждёт ответа в диалоге и после подтверждения показывает SnackBar
565d414 Выход из профиля теперь спрашивает подтверждение и только потом возвращает на экран входа
cc08f85 Сумма и комиссия в ордере пересчитываются при вводе, после отправки показывается SnackBar
fd24bff Добавил проверки ордера: хватает ли денег на покупку, не больше ли продаём чем есть, лимитная
цена больше нуля
f537559 Заработали переключатели покупки/продажи и типа ордера, лимитная цена и срок действия
доступны только для лимитного
7e27075 Форма ордера переведена на Form с контроллерами, в количество можно вводить только цифры
a0aa53b Добавил экран регистрации с проверкой полей и совпадения паролей, открывается с экрана входа
0653ee1 Добавил кнопку показа пароля, а «Забыли пароль?» теперь показывает SnackBar
b14620e Вход теперь проверяет email и пароль перед переходом, проверки вынес в validators.dart
a20c2f6 Карточка акции и ордер стали под-маршрутами рынка и портфеля, нижняя панель теперь не
пропадает
a7c3af9 Вкладки перенёс на StatefulShellRoute, вместо HomeShell теперь AppShell без своего состояния
25dbb27 Убрал Navigator.push, карточка и ордер открываются через context.push, а выход через go
46750f6 Подключил MaterialApp.router, вход теперь переходит на рынок через go
a7faff1 Добавил router.dart, у каждого экрана теперь есть свой адрес
3732338 Карточка акции и ордер теперь получают id и сами находят акцию, готовлю их к адресам
6081bbf gitlog.txt больше не попадает в репозиторий, это файл для отчёта
28b7b2a L3: добавлен пакет go_router
fea5d6a Прокрутка: свечение у края вместо растягивания
ff2eb2e Правки UI: аватары с тикером, склонение «сделок», перенос суммы
b7aeb9f Тема: схема fidelity, синий стал насыщеннее
7ad3a18 Навигация для демонстрации: NavigationBar
95dca53 Профиль: статистика, настройки, сброс портфеля
ed28f8b Рейтинг участников, своя строка выделена
83a948e Лента новостей с фильтром по рынку
0120ae4 Экран портфеля: сводка, распределение, позиции и ордера
97ac02e Форма ордера: тип, количество и расчёт с комиссией
5d2e202 Карточка акции: график, позиция, новости компании
1068cbc Экран рынка: поиск, фильтры и список акций
d1d7f75 Экран входа: статические поля и кнопка
9339254 Виджеты: StockTile, PriceChangeBadge, NewsCard, TickerAvatar, StatTile
723a097 Mock-данные: акции, позиции, ордера, новости, рейтинг
ebd5ef0 Тема Material 3: ColorScheme.fromSeed и цвета роста/падения
208e68f flutter create
```

8. Вывод

В TradeArena реализовано общее состояние на Provider и ChangeNotifier, интерфейс Repository, mock с задержкой и управляемой ошибкой, три состояния загрузки и retry. Форма ордера действительно записывает данные: покупка, продажа, отмена и сброс изменяют все подписанные экраны. Демонстрационный источник изолирован от UI; следующий этап может заменить его реализацией backend без переписывания экранов.